# 6CS014 – Complex Systems

## Module Assessment

| | |
|---|---|
| **Module Code** | 6CS014. |
| **Module Leader** | Thomas Hartley. |
| **Semester** | 1. |
| **Year** | 2023-24. |
| **Assessment** | Portfolio. |
| **% of module mark** | 100%. |
| **Set Date** | Teaching week 1. |
| **Due Date** | The deadline for the portfolio is listed below. |
| **Hand-in – what?** | The requirements for the portfolio are outlined below. |
| **Hand-in- where?** | Reports: Via Canvas.<br>Software: Via Canvas. |
| **Pass mark** | 40% grade required to pass the module. |
| **Method of retrieval** | Further work during resit period. Please see the resit advice for more information. |
| **Feedback** | Provisional grades and feedback will be available to students within 4 weeks of submission via Canvas. |
| **Collection of marked work** | Available within 4 weeks of the submission date via Canvas. |
| **University Regulations** | University's Academic Regulations: https://www.wlv.ac.uk/about-us/governance/legal-information/policies-and-regulations/academic-regulations/<br><br>Academic Integrity Policy: https://www.wlv.ac.uk/media/departments/office-of-the-vice-chancellor/documents/Academic-Integrity-Policy-from-2019-20.pdf<br><br>Performance descriptors are linked here: Level and Mark Descriptors |
| **Support** | Student Support and Wellbeing: https://www.wlv.ac.uk/current-students/student-support/student-support-and-wellbeing-ssw/<br><br>Study Guides: https://www.wlv.ac.uk/lib/skills-for-learning/study-guides/<br><br>Online Student Handbook: https://www.wlv.ac.uk/current-students/student-handbook/<br><br>Assessment information: http://www.wlv.ac.uk/assessment<br>You should also refer to your Course and Module Guides |

## Learning Outcomes

At the completion of the module, the student is expected to be able to:

1. Analyse and model complex systems.
2. Design and develop software for the control of complex systems.
3. Evaluate complex systems with respect to speed, accuracy and other factors applicable to the problems being focussed on.
4. To demonstrate an awareness of the issues involved in controlling emerging autonomous technologies, robots and other devices.

## Introduction and Assignment Overview

Complexity science is the scientific study of complex systems. These are systems with many parts that interact to produce global behaviour. These systems cannot easily be explained in terms of interactions between the individual elements. Complex systems are dynamic, unpredictable and have many elements. They exhibit emergent behaviour (rather than prescriptive behaviour) that is not easy to predict.

Complex systems can be found almost everywhere, examples include:

- Ecosystems,
- Cities and Businesses,
- Crowd Simulation,
- Our Brains.
- The Internet (not individual computer system).

In this module, you will explore and model complex systems. You will learn how to simulate (e.g., program) complex systems and research the topic in terms of artificial intelligence and emerging autonomous technologies.

The module assignment involves a portfolio of work to be **individually** completed. The portfolio comprises of two tasks and provides overall evidence of your development in the area of complex systems. The portfolio should include an application and a research report. **Please see the assignment specification below for more details.** The purpose of the assignment is to allow students to explore automated agents, software simulation of complex systems and artificial intelligence topics.

><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><

## Assignment Details

### Portfolio Task 1: Design and Build an Automated Agent

**Indicative portfolio task weighting:** 60%.
**Due date:** 22/12/23 – Teaching week 12, Friday.
**What to submit?:**

An **individual** software development project. See below for more details. You are also required to submit a short video of your agents implementing their behaviour.

**Word limit:** N/A.

For this portfolio task you are required to design and build a simulation (i.e., a virtual environment) and an automated virtual agent (i.e., robot that uses the virtual environment) that will act and react to its environment. By the end of this task, you will have created a software simulator that simulates a virtual environment.

You should build and document an autonomous agent written using the module tools, namely, Unity and C#. Below is a detailed overview of key elements of this portfolio task. This task is split into three parts. Each part requires an agent to be developed (i.e., coded).

### Problem / Application Area / Scenario for the Intelligent Virtual Agent

In the future robots or drones may deliver or collect things (e.g., parcels or people). In this assessment you are required to develop an autonomous automated agent that finds routes through an environment. **You are encouraged to apply a problem / application area / scenario or theme of your choice as a basis for the simulation and virtual agent.**

Here are some examples of possible agent problems / application areas / scenarios / themes that you could use:

- Office delivery or cleaning robot that needs to navigate to rooms in a building.
- Humanoid agent (e.g., post person) delivering or collecting parcels.
- Vehicle (e.g., lorry) agent delivering or collecting parcels.
- Drone agent delivering parcels.
- Taxi driver picking up and dropping off passenger(s).
- Animal storing food at locations for the winter.
- Lumberjack cutting down trees and returning to a sawmill.
- Animal collecting food and returning to a home.

**Tip:** If you are employed, your work may have an interesting problem that can be solved by a route-finding problem-solving agent. You could use this as a basis for your simulation.

Your also need to create a virtual environment that aligns to your agent problem / application area / scenario / theme. For example:

- Indoor office building.

- Small outside area with house or drop-boxes/points.
- Larger outside area with roads, buildings, or drop-boxes/points.
- Forest with trees and sawmill.
- Land animal (e.g., ants or tigers) environment, home location and food locations.
- Water (e.g., fish) animal environment, home location and food locations.
- Air animal (e.g., birds) environment, home location and food locations.
- Medieval (e.g., Footman collecting food).
- Futuristic (e.g., robots).

You are encouraged to apply an interesting problem / application area / scenario / theme as a basis for your agent and environment; however, don't worry if you cannot think of anything. A straightforward robot delivery problem and environment is fine. In fact, there is lots of scope to be creative with this type of application area.

**In addition, you must only implement <u>one</u> type of agent (e.g., delivery or collection) for all parts of the assignment. <u>You must keep the agent type and environment theme the same for all parts of the assignment.</u>**

## <u>Part 1 - Virtual Agent for a Known Environment and Single Delivery or Collection Location</u>

You are required to develop an autonomous intelligent agent that solves a problem (e.g., delivering parcels) using route-finding. The agent should begin at a start location, move to a goal location, and then return to the start location.

The agent you create should be able to navigate a **waypoint graph** representation of your virtual environment (see the "Agents Virtual Environment Representation" section below for details). The agent should be able to move from a start location and navigate (using the A* pathfinding algorithm) to another node on the waypoint graph (e.g., a delivery location). When the agent has reached their target, they should return to the start location.

Once core functionality has been implemented you should enhance the agent by extending its abilities. For example:

- navigate to more locations once the agent has returned to the start location.
- goal location(s) could be selectable in the Unity editor or at runtime (e.g., improve the robustness of the implementation). The agent could find the goal location(s) in the scene and move to one, then move back to the start.
- your agent could have multiple start locations (e.g., improve the robustness of the implementation). The agent could move to a goal and back to the start. It would then move to the next start location and move to the goal.
- your agent could respond to events in the environment (e.g., stop at traffic lights or unforeseen obstacles in its way).

- you could extend or create a new path cost and/or heuristic function. For example, you could incorporate time in the path cost and heuristic calculation. This would allow the agent to consider time taken as part of its problem-solving process. This type of problem-solving agent should be able to find routes that are both short in distance and take the least amount of time to navigate.

Higher grades will be awarded for extensions that are more advanced. For example, extending the path cost heuristic to monitor travel time and change agent routes would achieve a higher grade than a simple system that pauses the agent when it is in proximity of an obstacle (e.g., dog crossing the road).

The agent should move through the environment; however, additional actions, such as the act of delivering or collecting a parcel do not need to be animated / virtually recreated. For example, in a delivery scenario you can simply move to a location and move back to the start again. If you want to be more sophisticated, you could animate a box/parcel in Unity that disappears into the delivery location. However, this is not essential.

The agent in this task has full knowledge of the virtual environment. The agent can see the whole environment and should use path-finding techniques to navigate the environment. **You should implement the world representation (waypoint graph / map) and pathfinding algorithm (A*) yourself using the techniques introduced during the module. You should <u>not</u> use navigation methods built into Unity.**

You also need to output performance information (see the "Measuring Performance" section below.)

**Also, please prefix your Unity scene name and main agent C# class name with Part1_.** For example, your scene could be called Part1_Town and your agent could be call Part1_Agent.

## <u>Virtual Environment Representation for the Pathfinding A* Algorithm</u>

**You need to create a waypoint graph representation of the environment that your agents will use to navigate to locations.** Your waypoint graph should have at least **10 nodes**. You should create a **<u>standard waypoint graph</u>** with **at least 10 nodes** and a reasonable set of connections (i.e., edges) between the nodes. The graph should be able to work with the standard pathfinding algorithm A*. A user should be able set a start location and **<u>may</u>** be able to set multiple goal locations (**up to 5 goal locations**). A user should be able to do this via an interface or the Unity Inspector. The start location cannot be a goal location.

**As noted above, you should implement the world representation (waypoint graph / map) and pathfinding algorithm (A*) yourself using the techniques introduced during module. You should not use navigation methods built into Unity.**

**Measuring Performance**

The performance of the agent you create should be measured during all their runs / trips / journeys. Performance measures should include time and total distance. During the simulation, a timer should be used to time the duration for the total task time. The output from performance measures can be displayed via a user interface or printed to the Unity console.

**Part 2 - Collaborative Virtual Agent for a Known Environment and Single Delivery Location with Multiple Items**

The second agent builds upon the work you completed for the first agent. You should use the same agent and environment theme. However, note in this scenario the agent has different abilities / behaviour. **Therefore, you should make a copy of your scene and agent from part 1 and adapt it. Or copy your scene and start your agent from again scratch using the knowledge you have gained from part 1.**

The agent in this part should have the same abilities as part 1; however, the agent should also be able to act and re-act to obstacles and other instances of itself in the environment.

In addition, the agent should be able to carry up to 10 items (e.g., parcels). The number of items an agent is going to carry should be able to be set in the Inspector or via a user interface. The speed of the agent should be affected by the number of parcels it is carrying. The speed of the agent should drop 10% for each parcel the agent is carrying. The speed of the agent should drop a maximum of 90% its original speed. The agent should also include some sort of visual representation of the items it is carrying. For example, you could add a 3D model for each parcel the agent is carrying, or you could simply output a number above the agent to indicate the number of parcels it holds.

**The agent should monitor its environment for other agents (i.e., another instance of itself).** The agent should collaborate with other agents (i.e., other instance of itself) by monitoring its separation and speed. The agent should adjust its speed (generally slow) when other agents come within a safety zone. If other agents get too close the agent should stop. The agents should communicate and collaborate / negotiate in order to allow one agent to keep moving. You don't want both agents to stop and continually block each other and the waypoint map forever. You should be able to add at least three instances of the same agent (i.e., game object / class) into the environment.

Your collaboration algorithm should consider the agents speed and immediate direction / destination when deciding which agent should move first / have right of way. Remember, your agents speed should slow based on the number of items it is carrying. Therefore, you want to avoid a slow agent blocking a faster agent.

Once agents have interacted and collaborated, they should adjust their outward behaviour to illustrate how the collaborated. For example, an agent

could stop before a waypoint node to let another agent reach the node and travel down a different connection or the agents could rerun pathfinding to find alternative routes. Higher grades will be awarded for agents that have robust and sophisticated collaboration algorithms and outward behaviours.

Don't forget to include performance information (see above) in the agent code. You should also output information about collaboration encounters between agents. **Please prefix your Unity scene name and main agent C# class name with Part2_.** For example, your scene could be called Part2_Town and your agent could be call Part2_Agent.

## Part 3 - Agent for a Known Environment and Multiple Delivery or Collection Locations

The third agent explores the scenario of multiple delivery locations. This agent makes use of techniques you completed for the first and second agents; however, it will have a different structure. You are required to develop an autonomous automated agent that represents a delivery **or** collection agent (e.g., robot) that has **multiple** delivery or collection locations. The agent should start at a home location and deliver **or** collect items (e.g., parcels) from multiple goal locations. The agent should be able to act and re-act to obstacles and other instances of itself in the environment.

The agent should be able to deliver or collect an item (e.g., parcel or food) from up to **10 goal locations**. This means you should be able to set up to **10 goal locations** (e.g., delivery or collection locations) in your waypoint graph.

The agent should plan the shortest route that starts at the start location (e.g., delivery depot), visits each delivery or drop-off location and returns to the start location.

This type of problem is time-consuming to solve using traditional pathfinding techniques. It would take a long time to calculate the overall shortest route. Therefore, the agent should use the following technique to solve this problem:

- Ant Colony Optimisation.

**You should implement the Ant Colony Optimisation algorithm yourself. You should not use prebuilt tools to implement the algorithm. If you do use prebuilt, you will be capped at a pass for the Ant Colony Optimisation elements of the assignment.**

You should still use A* pathfinding to determine routes / connections between goal (e.g., delivery) locations. **The agent should also have the collaborative abilities of the agent you developed for part 2.** You should be able to add at least three agents into the environment.

The user should be able to set the values for key properties in of the Ant Colony Optimisation algorithm in the inspector or via a user interface. For example, Q, alpha and beta.

Don't forget to include performance information (see above) in the agent code. You should also output information about collaboration encounters between agents. **Please prefix your Unity scene name and main agent C# class name with Part3_.** For example, your scene could be called Part3_Town and your agent could be call Part1_Agent.

**Video of the Agent**

**For this portfolio task you are required to submit three screen capture videos demonstrating all agent actions for each part. The first video file name should be prefixed with part1_, the second part2_ and the third part3_.**

**The videos should be no longer than 180 seconds. Please do not zip the video file when you submit it.** You should use video capture packages, such as OBS (https://obsproject.com/), to record the video of your development plan. **If you have any questions or concerns about this, please speak to a member of the module team.**

**For this portfolio learning task, you may be required to demonstrate the agents you have developed to a member of the module team. This may happen for a number of reasons; for example, if you have not named your Unity scene name and main agent C# class clearly.** If asked to demo you should come prepared to present a live demo of your agents and answer questions related to their development. For example, your applications and agents' structure, understanding of techniques and principles used.

**What to Hand-in?**

When you have completed the work, you should submit four files through the portal on Canvas. Please submit the following:

- A .zip file containing your Unity project.
- Three screen capture videos demonstrating all agent actions. One video for part 1, another for part 2 and a third for part 3. Each video should be no longer than 180 seconds. Please do not zip these files.

><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><

**Portfolio Task 2: Investigation of Complex Systems with Respect to Artificial Intelligence and Emerging Autonomous Technologies**

**Indicative portfolio task weighting:** 40%.
**Due date:** 22/12/23 – Teaching week 12, Friday.
**What to submit?:** An **individual** academic **report**.
**Word limit:** Approx 2,000 words.

The aim of portfolio task 2 is to learn about and demonstrate evidence of learning about complex systems with respect to artificial Intelligence and emerging autonomous technologies. You are required to research a complex systems topic and present your findings in the form of an academic report.

Below are examples of possible report topics. **You can select your own title; however, you must get your title agreed by a module tutor.**

Examples of research report topics:

- Deep neural nets
- Autonomous flying vehicles
- Smart robots
- Autonomous space rovers
- Virtual assistants
- Etc.

You should produce a report of **approximately 2500 words** (about four pages of A4 using a 12pt. font, not including figures). The report should contain the following sections:

- **Introduction** – this should include an introduction to your chosen topic and explain why it is important. It should, in straightforward style state the purpose of the report / the problem being investigated.
- **Aim and Objectives** – include a clear section the outlines the aims and objectives of the report.
- **Literature Review** - An in-depth investigation of your chosen topic area, which draws upon a wide range of academic and research-based sources.
- **Analysis of Findings** – An analysis and evaluation of the research found during the literature review. This should be related to the aims of the report.
- **Conclusion** - A conclusion that draws out your findings.
- **References** - A list of references. Correct use of the Harvard referencing system.

**What to Hand-in?**

When you have completed the work, you should submit word file containing your report through the portal on Canvas. Please do not zip this file.

><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><

**General Important Information about the Assignment**

Please read the key points below:

1. You must use the prescribed module tools to develop your game. These are: **Unity version 2022.*.*f1 and C#.** If you do not use these tools, you may fail the module.

2. You should use resources on the web to support your learning and development as a game's developer / programmer. However, you are **not permitted to copy online resources in part or entirety and submit them as your work**. This also includes following online tutorials in part or entirety and submitting them as your work. **The agents you create for**

**this portfolio should be designed and developed specificity for this portfolio. You should start your development from an empty Unity project and create code based on your own knowledge and understanding. However, you should/can use pre-made graphical assets.** Please see the information at the end of this document and follow this link for more information regarding academic misconduct: http://www2.wlv.ac.uk/webteam/curr_sdts/sharpen/ss-HowtoAvoidAM.pdf.

3. You should use a new project or template project for your game. Do not reuse old projects or projects from other work.

4. **You must create a game in a 3D environment**, you cannot use Unity's 2D game systems.

5. You are only allowed to use freely available assets that can be freely accessed during the module run (i.e., no free assets that are not currently available). You should include the source of assets in the "Assets Used" document. You should not use any paid assets.

6. You are not allowed to use paid for tutorials or learning resources in any form.

><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><

## Marking Schemes

**The marking schemes / grids for each portfolio task can be found in the same folder as the assessment on Canvas.**

Please read the marking grids carefully. Please speak to the module leader if you have any questions.

Remember – *Grades are not mechanically calculated but are a matter of academic judgement based on the marking criteria.*

You should be able to find the following marking grids for this portfolio:

- Marking Grid - Portfolio Learning Task 1: Simulator and Agent (Indicative weighting 60%).
- Marking Grid - Portfolio Learning Task 2: Investigation of Complex Systems (Indicative weighting 40%).

><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><><